LECTURE 1

Developing LLM applications on Databricks

14 October 2026

16:00–18:00 Amsterdam



Agenda

What we will cover today

01 **Development options**
Tools, authentication, and serverless environments

02 **Project overview**
Project configuration and `pyproject.toml`

03 **Databricks bundles**
Package resources and run notebooks as jobs

04 **Foundation models**
Model services and the gateway interface

05 **Provisioned throughput**
Dedicated capacity and reservation options

06 **External models**
Provider access, governance, and routing

07 **Example code**
Retrieve arXiv metadata, write a Delta table, and review an AI-assisted job configuration



The course development environment

Local code, with execution on Databricks

Python 3.12	The project pins Python to `3.12.*` and declares its dependencies in `pyproject.toml`.
Environment 5	The repository selects serverless environment version **5**, with Databricks Connect **18.0**.
Application settings	`project_config.yml` keeps the application's configuration alongside its code.
Installed packages	Check the active interpreter and `python -m pip list` when troubleshooting dependency differences.



Serverless environment versions

The course version and the latest release can differ

Environment	Python	DB Connect	Support until
6 / latest	3.12.3	19	3 Sep 2029
5 / course	3.12.3	18	25 Feb 2029
4	3.12.3	17.3.2	13 Aug 2028

> An environment version fixes a client API and package baseline. Keep local dependencies aligned with the environment your workload uses.

Environment version reference · Checked 13 October 2026



Databricks development tools

Different tools for different parts of the workflow

Tool	What it does	In this course
Databricks CLI	Authenticates and automates workspace operations from a terminal.	Profiles, bundle validation, deployment, and job runs
Databricks Connect	Runs Spark workloads on Databricks from local Python code.	Local development and debugging with remote compute
VS Code extension	Connects the editor to a Databricks workspace and development workflow.	Project setup, code synchronization, and remote execution



Authentication using the Databricks CLI

Browser-based OAuth login for local development

INSTALL ON macOS

```
brew tap databricks/tap
brew install databricks
```

LOGIN AND CHECK PROFILES

```
databricks auth login \
  --host <workspace-url> \
  --profile course

databricks auth profiles
```

Never paste credentials into slides, source control, or shared notebook outputs.

A named profile

Complete the browser sign-in, then use the same profile with your development tools.

The SDK can use configured Databricks authentication. An OpenAI client needs a compatible token or authentication adapter.

No OpenAI API key is needed to call the course's Databricks model service.



Foundation models

Foundation models and Unity Gateway

A governed model service separates application code from its backend

THE SERVICE OUR APPLICATION CALLS

`course_ops.gateway.chat`

Pay-per-token

Databricks serves the model from shared capacity.

Provisioned throughput

A serving endpoint provides dedicated capacity.

External provider

A provider service connects to a model outside Databricks.

> The model service handles routing. Its destination determines which model and capacity serve the request.

Calling the course model service

OpenAI Python client, authenticated with Databricks

```python
from databricks.sdk import WorkspaceClient
from openai import OpenAI

workspace_client = WorkspaceClient()
databricks_token = workspace_client.tokens.create(lifetime_seconds=1200).token_value
client = OpenAI(
    api_key=databricks_token,
    base_url=f"{workspace_client.config.host}/ai-gateway/mlflow/v1",
)
response = client.responses.create(
    model="course_ops.gateway.chat",
    max_output_tokens=256,
    input="What is Databricks?",
)
print(response.output_text)
```

WorkspaceClient uses the configured workspace host and SDK authentication. Token creation permission is required; the token expires after 20 minutes.



AI Gateway benefits

Governance around model requests and responses

Usage and cost

Monitor token consumption and attribute activity to services, users, or projects.

Traffic management

Configure rate limits, traffic splitting, and fallback destinations.

Payload logging

Enable request and response logging for debugging and audit workflows.

Guardrails

Apply service policies to evaluate request content and model responses.

Configure and verify these features for your service. Logging and policies are not implied by using a gateway URL.



Databricks-hosted models

Pay-per-token access to shared model capacity

Useful for

- Experimentation and exploration
- Development and testing
- Intermittent application traffic
- Comparing model behavior

Operating considerations

- Token-based usage charges
- Shared capacity and rate limits
- Model and regional availability
- Access through service permissions

> No dedicated capacity to provision. Your identity still needs access to the model service.



Provisioned throughput

Dedicated capacity has more than one purchasing model

On-demand

No reservation term. Change or remove capacity as needed.

Notebook 1.2 creates a GPT OSS 120B on-demand endpoint, then a model service that routes to it.

SDK/API deployment is available.

Reserved

Commit for **1 or 3 months**, starting at **50 model units**, in increments of 50.

Eligible models are limited. Create a new reserved endpoint, then route a model service to it.

Existing endpoints cannot convert.

> Active reservations cannot be canceled or their endpoints deleted. A short lab cleanup cannot undo that commitment.

Sizing provisioned throughput

Workload estimate, supported allocation, then performance testing

01 / Describe demand

Example: 60 requests/min, each with 2,000 input and 300 output tokens.

120,000 input + 18,000 output tokens/min

02 / Check supported capacity

Notebook 1.2 checks GPT OSS 120B eligibility and reads `model_unit_chunk_size` from the optimization API.

03 / Allocate and estimate cost

The lab sets `CAPACITY_CHUNKS = 2`, allocates **2 × chunk size**, and estimates DBUs for that allocation.

04 / Validate under load

Test realistic traffic against a response-time target. Measure latency and errors, then adjust capacity and retest.

Two chunks are a lab setting, not a sizing recommendation. Notebook 1.2 demonstrates deployment and inference, not load testing.

External models

Models hosted outside Databricks, accessed through the gateway

Provider connection	A model provider service connects to an external API using provider credentials.
Governed access	The application calls a Databricks model service with Databricks authentication.
Provider behavior	Supported modalities, parameters, and limits depend on the chosen provider and model.
Operational checks	Review network access, credentials, provider charges, and enabled gateway features.



Model hosting comparison

The hosting choice sits behind the model-service interface

Consideration	Pay-per-token	Provisioned	External provider
Capacity	Shared	Dedicated	Provider-managed
Cost basis	Token usage	Allocated capacity or reservation	Provider pricing and applicable gateway charges
Setup	Model access and permissions	Endpoint provisioning and service routing	Provider connection and credentials
Typical use	Exploration or variable traffic	Sustained demand with capacity planning	A specific provider or model requirement

Check eligible models and workspace availability rather than relying on a static model catalog.



Gateway capabilities and scope

The legacy endpoint feature matrix describes a different surface

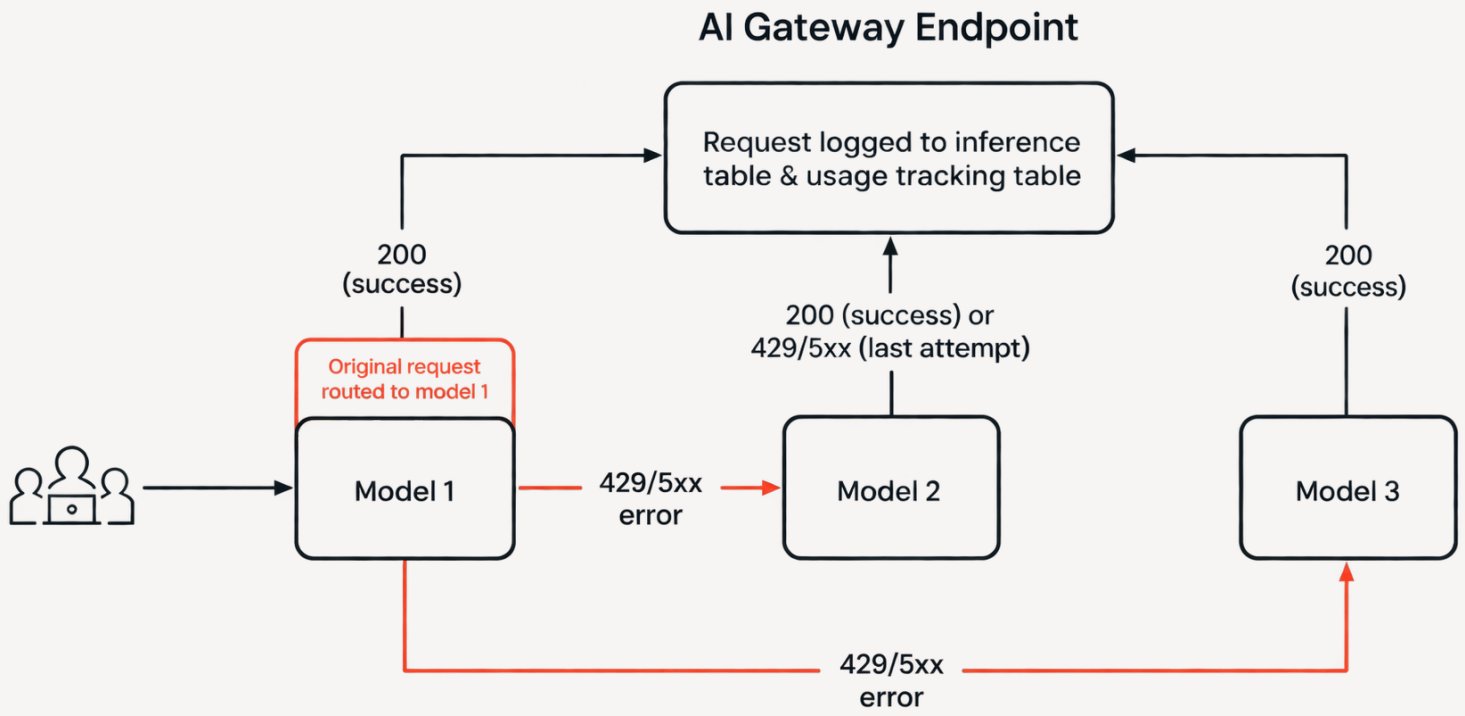
Capability	Configuration or check
Access	Unity Catalog privileges on the service and related objects
Routing	Model-service destinations, traffic weights, and fallbacks
Observability	Usage records and configured inference logging
Guardrails	Attached service policies and preview availability
Model features	Provider, destination model, API, and regional support

Unity Gateway documentation · Verify the actual service configuration during the demo.



AI Gateway fallbacks

Backup destinations handle failed primary requests



Routing behavior

Traffic weights choose the initial destination.

Failures can trigger retries against an ordered fallback list.

Verify behavior in usage records and test each backup model.

Conceptual fallback flow. In Unity Gateway, configure fallback order on the model service.

AI Gateway guardrails

Service policies govern request and response content

Sensitive information

Evaluate content for PII and apply the configured allow or deny decision.

Unsafe content

Use supported policies to check content risks, including prompt injection.

Organization rules

Attach custom policies for requirements such as eligible project tags.

Input and output

Policies can evaluate the request before invocation and the response afterward.

Service policies are Beta. Automated checks do not guarantee detection of every harmful or sensitive input.

Declarative Automation Bundles



Declarative Automation bundles

formerly Databricks asset bundles

Version application code and resource configuration together, then deploy them to a target workspace.

In this repository

`databricks.yml` defines the bundle and its deployment targets.

Other resource-management options include Terraform, the REST API, SDKs, the CLI, and MLflow APIs.



Resources managed with bundles

Support is broad. Ownership still matters.

Application resources

- Lakeflow Jobs and pipelines
- Apps, dashboards, and alerts
- Clusters and SQL warehouses
- MLflow experiments and registered models
- Serving endpoints and model services

Data and platform resources

- Catalogs, schemas, and volumes
- Lakebase database resources
- Quality monitors
- Secret scopes and related resources
- Provider and MCP services

Check resource support in your CLI version. Assign one deployment owner per resource to avoid conflicting updates.



Important bundle commands

Run from the bundle directory

`databricks bundle validate -t dev`	Validate the configuration
`databricks bundle deploy -t dev`	Deploy code and resources
`databricks bundle run -t dev <job-key>`	Run a configured job
`databricks bundle destroy -t dev`	Delete the target's deployed resources

> **Deployment and deletion affect real resources.**
>
> Review the target and planned changes before confirming either action.



Homework

Apply today’s workflow to your own project

01 Choose the use case you want to build.

02 Develop preprocessing in VS Code and run it on Databricks.

03 Experiment with LLM calls through a model service.

04 Submit a pull request with your code and observations.

Use the arXiv ingestion notebook as a reference. Keep credentials out of your pull request.



Questions?

Development, model access, or your project

Continue the conversation on Discord.